

Intermediate (make search conditional) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `search_tool_integration_practice.py` : this is the exact conditional edge the Build Task needs at its very first fork — should this task even touch the search tool, decided by the graph itself, before the model ever reasons about it. **If not:** the Build Task's router node would be the first place you ever wrote a real `should_search` function, with no smaller version to trust.

Read both depths — they're not "wrong, right," they're 2 real, valid ways to solve the same problem, with real tradeoffs between them.

Basic Version

Approach 1 — a keyword check

```
# search_tool_integration_practice.py – Intermediate section
def should_search(state):
    task = state["messages"][-1].content.lower()
    keywords = ["document", "policy", "according to", "the file says"]
    if any(keyword in task for keyword in keywords):
        return "search"
    return "skip"

graph.add_conditional_edges(
    "router", should_search, {"search": "search_node", "skip": "reason_node"},
)
```

Approach 2 — asking a small model to decide

```
# search_tool_integration_practice.py – Intermediate section
def should_search(state):
    task = state["messages"][-1].content
    prompt = (
        "Does answering this question require looking up information "
        "in a document set? Answer only yes or no.\n\nQuestion: " + task
    )
```

```
answer = model.invoke(prompt).content.strip().lower()
if "yes" in answer:
    return "search"
return "skip"
```

Both work. Approach 1 is free and instant but easy to fool with unexpected phrasing. Approach 2 catches more phrasings but costs one extra model call on every single task.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a keyword check, with type hints

```
# search_tool_integration_practice.py – Intermediate section
def should_search(state: dict) -> str:
    task = state["messages"][-1].content.lower()
    keywords = ["document", "policy", "according to", "the file says"]
    if any(keyword in task for keyword in keywords):
        return "search"
    return "skip"

graph.add_conditional_edges(
    "router",
    should_search,
    {"search": "search_node", "skip": "reason_node"},
)
```

Approach 2 — a small classification call

```
# search_tool_integration_practice.py – Intermediate section
from langchain_core.messages import HumanMessage

def should_search(state: dict) -> str:
    task = state["messages"][-1].content
    classification_prompt = (
        "Does answering this question require looking up information "
        "in a document set? Answer with exactly one word: yes or no.\n\n"
        f"Question: {task}"
    )
    answer = model.invoke([HumanMessage(content=classification_prompt)])
    return "search" if "yes" in answer.content.strip().lower() else "skip"

graph.add_conditional_edges(
    "router",
    should_search,
```

```
{ "search": "search_node", "skip": "reason_node"},
)
```

Difference from Basic: both approaches add full type hints. Approach 2's prompt is more precise about the expected answer format ("exactly one word: yes or no"), which makes the "yes" in answer check more reliable. Neither version yet mixes the two strategies — each one commits to keywords-only or model-call-only for every single task, which is what Approach 3 changes.

Approach 3 — a hybrid: cheap check first, model call only when ambiguous

Story: a keyword list and a single model call each fail in a different, predictable direction — keywords miss unexpected phrasings, a model call costs one extra round trip on every task, even the 90% that were obvious either way. **If not:** the Build Task's router would either misroute real search-needing questions or pay for a model call on every single task, including the ones any keyword list already handles for free.

```
# search_tool_integration_practice.py – Intermediate section
from langchain_core.messages import HumanMessage

KEYWORDS = ["document", "policy", "according to", "the file says"]

def should_search(state: dict) -> str:
    task = state["messages"][-1].content
    lowered = task.lower()
    has_keyword = any(keyword in lowered for keyword in KEYWORDS)

    # confident hit: a clear keyword match
    if has_keyword:
        return "search"

    # confident skip: short, no keyword, no question mark
    looks_conversational = len(task.split()) <= 6 and "?" not in task
    if looks_conversational:
        return "skip"

    # ambiguous: fall through to the more expensive, more accurate check
    classification_prompt = (
        "Does answering this question require looking up information "
        "in a document set? Answer with exactly one word: yes or no.\n\n"
        f"Question: {task}"
    )
    answer = model.invoke([HumanMessage(content=classification_prompt)])
    return "search" if "yes" in answer.content.strip().lower() else "skip"
```

```
graph.add_conditional_edges(
    "router",
    should_search,
    {"search": "search_node", "skip": "reason_node"},
)
```

Most real questions hit one of the two confident branches and never pay for a model call at all — the extra round trip is spent only on the genuinely ambiguous middle, like "what's our time-off rule?", which has no keyword and isn't obviously conversational either.

Approach 4 — logging every routing decision, for later review

Story: a hybrid router is still a guess, and guesses are worth checking — this keeps Approach 3's logic unchanged, but records which branch fired and why, so misroutes can be found and fixed later instead of going unnoticed. **If not:** "which cases are being misrouted" would stay a question you could only answer by inspection, never by evidence.

```
# search_tool_integration_practice.py – Intermediate section
import logging

logger = logging.getLogger("routing")

def should_search(state: dict) -> str:
    task = state["messages"][-1].content
    lowered = task.lower()
    has_keyword = any(keyword in lowered for keyword in KEYWORDS)

    if has_keyword:
        logger.info("routed=search reason=keyword task=%r", task)
        return "search"

    looks_conversational = len(task.split()) <= 6 and "?" not in task
    if looks_conversational:
        logger.info("routed=skip reason=conversational task=%r", task)
        return "skip"

    classification_prompt = (
        "Does answering this question require looking up information "
        "in a document set? Answer with exactly one word: yes or no.\n\n"
        f"Question: {task}"
    )
    answer = model.invoke([HumanMessage(content=classification_prompt)])
    decision = "search" if "yes" in answer.content.strip().lower() else "skip"
```

```
logger.info("routed=%s reason=model_call task=%r", decision, task)
return decision
```

Nothing about the routing behavior changes — this is purely observability. Reviewing these logs later is how you'd actually discover, with evidence instead of a guess, whether the keyword list is missing real phrasings or the model call is being triggered far more often than expected.

Difference from Approach 1, and between Approaches 3/4: Approach 1/2 each commit to one strategy for every task — either always the cheap check (missing some real search-needing questions) or always the model call (correct more often, but paying the cost and latency on every task, including the 90% that were obvious either way). Approach 3 fixes that by only paying for the model call on the genuinely ambiguous cases. Approach 4 doesn't change the routing decision at all — it adds a record of *why* each decision was made, which is what actually lets you improve the keyword list or the conversational heuristic later, based on real misroutes instead of guessing at them.

Which one should you actually write? Start with Approach 1 (keywords only) — it's free, instant, and good enough while you're building and testing the rest of the graph. Move to Approach 3's hybrid once you've actually seen the keyword version misroute a real question in testing — don't add the model-call fallback speculatively before you've confirmed you need it. Add Approach 4's logging once this router is handling real, varied traffic (not just your own test questions) — that's the point where "which cases are being misrouted" stops being something you can answer by inspection and becomes something you need evidence for.